

Guía de Consulta — Validación de datos con `express-validator`

Sigue el mismo ejemplo de la guía anterior (Artistas, Canciones, Playlists). Aquí solo trabajamos sobre `Artista` para que quede claro el patrón completo; ustedes deben replicarlo en las 5 colecciones de Numerología.

1. Teoría — ¿por qué no basta con la validación de Mongoose?

El schema de Mongoose (`required`, `enum`, `type`) valida datos **justo antes de guardar en la base de datos**. El problema es que para ese punto la petición ya pasó por todo el controller, y si algo está mal, te enteras con un error crudo de Mongoose (`ValidationError`) que:

- No controla el **formato** de la respuesta HTTP
- No valida cosas que Mongoose no sabe hacer bien (ej. "email válido", "fecha no futura", "longitud mínima de texto")
- No te deja **sanitizar** (limpiar/normalizar) el dato antes de usarlo

`express-validator` valida **en la capa de rutas**, antes de que la petición llegue al controller. Si algo está mal, cortas ahí mismo con un 400 claro y estructurado, y el controller nunca se ejecuta con datos sucios.

```
Cliente → Ruta → [express-validator] → Controller → Modelo (Mongoose) → DB
                        ↑
                    si falla, responde 400 aquí y no sigue
```

Regla mental: Mongoose es la última línea de defensa (protege la base de datos). `express-validator` es la primera línea de defensa (protege tu API y le da mejores mensajes de error al cliente).

2. Instalación

```
npm install express-validator
```

```
// package.json
"dependencies": {
  "express": "^4.19.2",
  "express-validator": "^7.1.0",
  "mongoose": "^8.5.0",
  "dotenv": "^16.4.5"
}
```

3. Los 3 bloques que vas a usar siempre

`express-validator` trabaja con **middlewares encadenados**. Tres piezas:

Pieza	Qué hace	De dónde se importa
body(), param(), query()	Define QUÉ campo validar y de dónde viene (body, params de la url, query string)	express-validator
Métodos encadenados (.notEmpty(), .isEmail(), .isLength(), etc.)	Las reglas de validación/sanitización, se encadenan con .	(vienen con el objeto anterior)
validationResult(req)	Recolecta los errores acumulados por las reglas de arriba y los devuelve	express-validator

4. Estructura de carpetas — dónde va la validación

```

proyecto-numerologia/
├── src/
│   ├── config/
│   │   └── db.js
│   ├── models/
│   │   └── Artista.model.js
│   ├── validators/                ← carpeta nueva
│   │   └── artista.validator.js
│   ├── middlewares/              ← carpeta nueva
│   │   └── validarCampos.js
│   ├── controllers/
│   │   └── artista.controller.js
│   ├── routes/
│   │   └── artista.routes.js
├── .env
├── package.json
└── index.js

```

- validators/ → contiene solo las **reglas** (qué se valida)
- middlewares/ → contiene el middleware **genérico y reutilizable** que revisa si hubo errores (no se repite en cada archivo)

5. src/middlewares/validarCampos.js — el middleware reutilizable

Este archivo se hace **una sola vez** y se reutiliza en todas las rutas del proyecto (Artistas, Canciones, Playlists, y luego en Numerología).

```

import { validationResult } from "express-validator";

// Revisa si las reglas del validator (que corrieron antes en la cadena)
// encontraron errores.
// Si hay errores, corta la petición aquí con 400. Si no hay errores, deja pasar
// con next().
export const validarCampos = (req, res, next) => {
  const errores = validationResult(req);

  if (!errores.isEmpty()) {

```

```

    return res.status(400).json({
      mensaje: "Error de validación",
      errores: errores.array().map((err) => ({
        campo: err.path,
        mensaje: err.msg,
      })),
    });
  }

  next();
};

```

6. src/validators/artista.validator.js — las reglas

```

import { body, param } from "express-validator";

// Reglas para CREAR un artista (todos los campos son obligatorios)
export const crearArtistaValidator = [
  body("nombre_artistico")
    .trim() // sanitización: quita espacios al inicio/final
    .notEmpty().withMessage("El nombre artístico es obligatorio")
    .isLength({ min: 2, max: 60 }).withMessage("El nombre debe tener entre 2 y 60 caracteres"),

  body("pais")
    .trim()
    .notEmpty().withMessage("El país es obligatorio")
    .isAlpha("es-ES", { ignore: " " }).withMessage("El país solo debe contener letras"),

  body("genero_musical")
    .optional() // el schema tiene default, no es obligatorio que lo envíen
    .isIn(["Reggaeton", "Salsa", "Pop", "Rock", "Vallenato"])
    .withMessage("Género musical no válido"),

  body("fecha_debut")
    .notEmpty().withMessage("La fecha de debut es obligatoria")
    .isISO8601().withMessage("La fecha debe tener formato válido (YYYY-MM-DD)")
    .custom((valor) => {
      // custom(): validación personalizada cuando los métodos por defecto no alcanzan
      if (new Date(valor) > new Date()) {
        throw new Error("La fecha de debut no puede ser futura");
      }
      return true;
    }),
];

// Reglas para ACTUALIZAR (los campos son opcionales, pero SI vienen deben ser válidos)
export const actualizarArtistaValidator = [
  body("nombre_artistico")
    .optional()
    .trim()
    .isLength({ min: 2, max: 60 }).withMessage("El nombre debe tener entre 2 y 60 caracteres"),

  body("pais")
    .optional()
    .trim()
    .isAlpha("es-ES", { ignore: " " }).withMessage("El país solo debe contener

```

```

letras"),

  body("genero_musical")
    .optional()
    .isIn(["Reggaeton", "Salsa", "Pop", "Rock", "Vallenato"])
    .withMessage("Género musical no válido"),
];

// Reglas para validar el :id que llega por params (se usa en GET/:id, PUT/:id, DELETE/:id)
export const idValidator = [
  param("id")
    .isMongoId().withMessage("El id proporcionado no es un ObjectId válido de MongoDB"),
];

```

Nota importante: aquí a propósito **no** incluimos `verificado` en ningún validator — recuerda la sección de mass assignment de la guía anterior. Si un campo no debe poder setearlo el cliente, ni siquiera le escribas una regla de validación; simplemente no lo destructures en el controller.

7. Controller — sin cambios en la lógica, solo se simplifica

Como la validación ya ocurrió en la ruta, el controller puede confiar en que `req.body` ya es válido:

```

// src/controllers/artista.controller.js
import Artista from "../models/Artista.model.js";

export const crearArtista = async (req, res) => {
  try {
    const { nombre_artistico, pais, genero_musical, fecha_debut } = req.body;
    const artista = await Artista.create({ nombre_artistico, pais, genero_musical, fecha_debut });
    res.status(201).json(artista);
  } catch (error) {
    res.status(400).json({ mensaje: "Error al crear artista", error: error.message });
  }
};

export const listarArtistas = async (req, res) => {
  try {
    const artistas = await Artista.find();
    res.status(200).json(artistas);
  } catch (error) {
    res.status(500).json({ mensaje: "Error al listar artistas", error: error.message });
  }
};

export const obtenerArtista = async (req, res) => {
  try {
    const artista = await Artista.findById(req.params.id);
    if (!artista) return res.status(404).json({ mensaje: "Artista no encontrado" });
    res.status(200).json(artista);
  } catch (error) {
    res.status(500).json({ mensaje: "Error al obtener artista", error: error.message });
  }
};

```

```

};

export const actualizarArtista = async (req, res) => {
  try {
    const { nombre_artistico, pais, genero_musical } = req.body;
    const artista = await Artista.findByIdAndUpdate(
      req.params.id,
      { nombre_artistico, pais, genero_musical },
      { new: true, runValidators: true }
    );
    if (!artista) return res.status(404).json({ mensaje: "Artista no encontrado" });
    res.status(200).json(artista);
  } catch (error) {
    res.status(400).json({ mensaje: "Error al actualizar artista", error: error.message });
  }
};

export const eliminarArtista = async (req, res) => {
  try {
    const artista = await Artista.findByIdAndDelete(req.params.id);
    if (!artista) return res.status(404).json({ mensaje: "Artista no encontrado" });
    res.status(200).json({ mensaje: "Artista eliminado correctamente" });
  } catch (error) {
    res.status(500).json({ mensaje: "Error al eliminar artista", error: error.message });
  }
};

```

8. Routes — donde se encadena todo

Aquí es donde se conecta: **validator** → **validarCampos** → **controller**, en ese orden exacto.

```

// src/routes/artista.routes.js
import { Router } from "express";
import {
  crearArtista,
  listarArtistas,
  obtenerArtista,
  actualizarArtista,
  eliminarArtista,
} from "../controllers/artista.controller.js";
import {
  crearArtistaValidator,
  actualizarArtistaValidator,
  idValidator,
} from "../validators/artista.validator.js";
import { validarCampos } from "../middlewares/validarCampos.js";

const router = Router();

router.post("/", crearArtistaValidator, validarCampos, crearArtista);
router.get("/", listarArtistas);
router.get("/:id", idValidator, validarCampos, obtenerArtista);
router.put("/:id", [...idValidator, ...actualizarArtistaValidator], validarCampos, actualizarArtista);
router.delete("/:id", idValidator, validarCampos, eliminarArtista);

export default router;

```

Por qué funciona el arreglo

`[...idValidator, ...actualizarArtistaValidator]`: Express acepta un array de middlewares en cualquier posición donde acepta un solo middleware, y los ejecuta en orden. Combinamos las reglas del `:id` con las del body porque el PUT necesita validar ambos.

9. Probar los casos de error (Postman/Thunder Client)

Caso 1 — falta un campo obligatorio:

```
POST /api/v1/artistas
{
  "pais": "Colombia"
}

// Respuesta 400
{
  "mensaje": "Error de validación",
  "errores": [
    { "campo": "nombre_artistico", "mensaje": "El nombre artístico es obligatorio" },
    { "campo": "fecha_debut", "mensaje": "La fecha de debut es obligatoria" }
  ]
}
```

Caso 2 — fecha futura (custom validator):

```
POST /api/v1/artistas
{
  "nombre_artistico": "Test",
  "pais": "Colombia",
  "fecha_debut": "2030-01-01"
}

{
  "mensaje": "Error de validación",
  "errores": [
    { "campo": "fecha_debut", "mensaje": "La fecha de debut no puede ser futura" }
  ]
}
```

Caso 3 — id inválido en la url:

```
GET /api/v1/artistas/123abc

{
  "mensaje": "Error de validación",
  "errores": [
    { "campo": "id", "mensaje": "El id proporcionado no es un ObjectId válido de MongoDB" }
  ]
}
```

10. Validación async — cuando necesitas consultar la base de

datos

Caso real: evitar que se registren dos artistas con el mismo `nombre_artistico`. Esto no lo puede validar una regla simple porque necesita ir a MongoDB.

```
// dentro de artista.validator.js
import Artista from "../models/Artista.model.js";

export const crearArtistaValidator = [
  // ...las reglas anteriores...

  body("nombre_artistico").custom(async (valor) => {
    const existe = await Artista.findOne({ nombre_artistico: valor });
    if (existe) {
      throw new Error("Ya existe un artista registrado con ese nombre artistico");
    }
    return true;
  }),
];
```

`express-validator` espera automáticamente a que el `custom()` `async` termine antes de seguir. No necesitas hacer nada especial en `validarCampos.js`.

11. Referencia rápida de validadores más usados

Método	Uso
<code>.notEmpty()</code>	Campo no puede estar vacío
<code>.isString()</code>	Debe ser texto
<code>.isEmail()</code>	Formato de correo válido
<code>.isLength({ min, max })</code>	Longitud de texto
<code>.isNumeric()</code> / <code>.isInt()</code> / <code>.isFloat()</code>	Números
<code>.isBoolean()</code>	true/false
<code>.isISO8601()</code>	Fecha en formato estándar
<code>.isMongoId()</code>	ObjectId válido de MongoDB (clave para validar <code>:id</code> y refs)
<code>.isIn([...])</code>	El valor debe estar en una lista permitida (equivalente al <code>enum</code> del schema)
<code>.isArray({ min })</code>	Debe ser un array, opcionalmente con mínimo de elementos
<code>.trim()</code>	Sanitización: quita espacios al inicio/final
<code>.escape()</code>	Sanitización: escapa HTML (previene XSS básico)
<code>.custom(fn)</code>	Regla personalizada (sync o async)
<code>.optional()</code>	El campo puede no venir; si viene, se valida

12. Checklist para aplicar esto en Numerología

- ☐ Crear `src/middlewares/validarCampos.js` (una sola vez, se reutiliza en todo el proyecto)
- ☐ Crear un archivo `*.validator.js` por cada colección: `usuario.validator.js`, `matriz.validator.js`, `prompt.validator.js`, `lectura.validator.js`, `bitacora.validator.js`
- ☐ En cada validator: reglas separadas para crear (todo obligatorio) y actualizar (todo `.optional()`)
- ☐ `idValidator` reutilizable con `.isMongoId()` para cualquier ruta con `:id`
- ☐ Para los campos que son ref a otra colección (ej. `usuario_id` en `MatricesNumerologicas`), validar con `.isMongoId()` y opcionalmente con un `.custom()` `async` que verifique que ese `_id` sí exista en la colección relacionada
- ☐ Ningún campo "protegido" (activo en `PromptsConfig`, cualquier flag administrativo) debe tener regla de validación en el validator de creación pública — ni siquiera se deestructura (ver guía anterior, sección 11)
- ☐ Cadena en las rutas siempre en el mismo orden: `[...validatorReglas]`, `validarCampos`, `controller`